

LABORATORY WORK #2

Normalization and DDL

Student: Aknazar Amirhan

Group: _____

Date: 21.09.2026

1. NORMALIZATION

Task 1. Normalization Forms

Original table

Booking_receipt(

booking_id,

passenger_full_name,

passenger_passport_number,

flight_number,

departure_airport_name,

departure_city,

arrival_airport_name,

arrival_city,

airline_name,

seat_numbers,

ticket_price

)

```
SQL Shell (psql)
ПРЕДУПРЕЖДЕНИЕ: Кодовая страница консоли (866) отличается от основной
страницы Windows (1251).
8-битовые (русские) символы могут отображаться некорректно.
Подробнее об этом смотрите документация psql, раздел
"Notes for Windows users".
Введите "help", чтобы получить справку.

postgres=# \c
Вы подключены к базе данных "postgres" как пользователь "postgres".
postgres=# \c airport
Вы подключены к базе данных "airport" как пользователь "postgres".
airport=# SELECT * FROM Booking_receipt;
 booking_id | passenger_fullname | passenger_passport_number | flight_number | departure_airport_name | departure_city | arrival_airport_name
-----
1 | Ali Friend | N123456 | KC101 | Almaty International Airport | Almaty | Astana International Airport
2 | Aruzhan Family | N234567 | KC102 | Astana International Airport | Astana | Almaty International Airport
3 | Dias Work | N345678 | FS201 | Almaty International Airport | Almaty | Frankfurt Airport
4 | John Smith | P456789 | LH500 | Frankfurt Airport | Frankfurt | Astana International Airport
5 | Maria Brown | P567890 | TM301 | Astana International Airport | Astana | Istanbul Airport
(5 rows)

airport=#
```

The original table violates 1NF because the seat_numbers attribute can contain multiple values in one cell, for example "12A, 12B". In 1NF, each attribute must contain an atomic value

TASK 3. Definition of functional dependencies (FD)

To justify the normalization steps, we identify the following functional dependencies (FD):

1. passenger_passport_number → passenger_full_name
2. departure_airport_name → departure_city
3. arrival_airport_name → arrival_city
4. flight_number → airline_name, departure_airport_name, arrival_airport_name
5. booking_id → flight_number, ticket_price
6. (booking_id, passenger_passport_number) → seat_number

TASK 2. Normalization to 1NF, 2NF, 3NF

1NF:

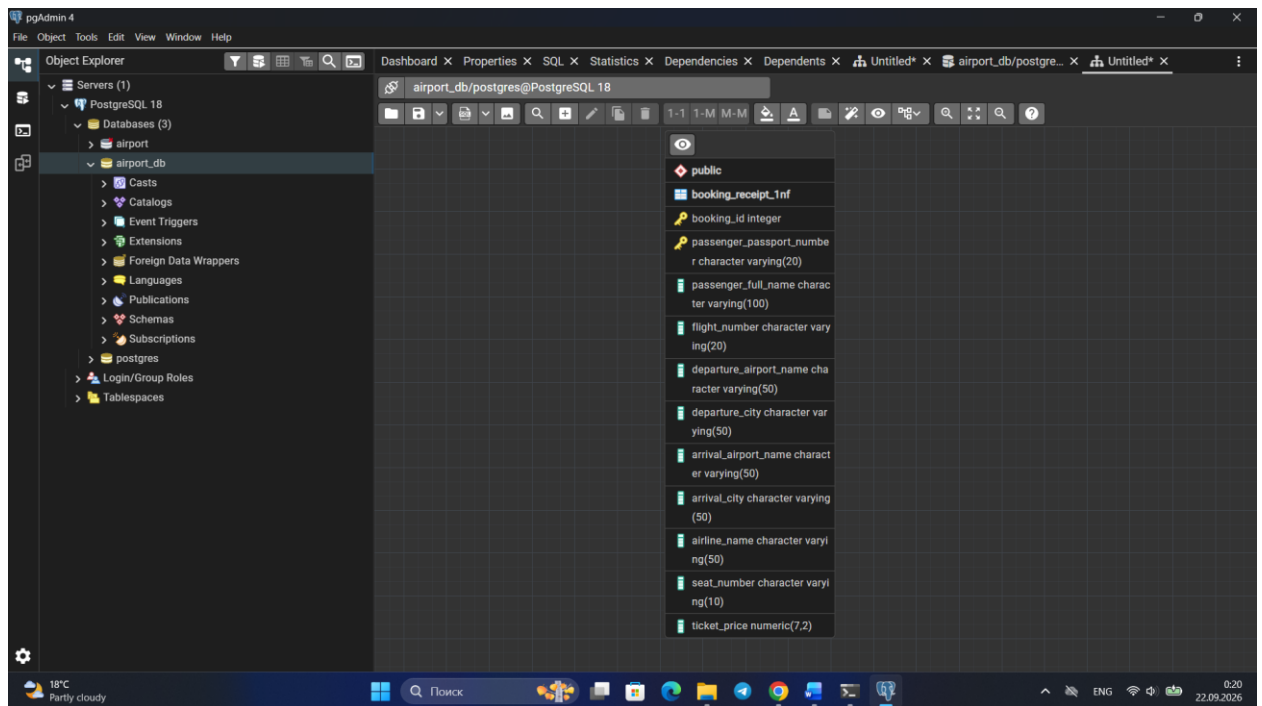
```

SQL Shell (psql)
1 | arrival_city | airline_name | seat_numbers | ticket_price
-----
1 | Astana | Air Astana | 12A, 12B | 50000.00
2 | Aruzhan Family | N234567 | KC102 | Astana International Airport | Astana | Almaty International Airport
3 | Dias Work | N345678 | FS201 | Almaty International Airport | Almaty | Frankfurt Airport
4 | John Smith | P456789 | LH500 | Frankfurt Airport | Frankfurt | Astana International Airport
5 | Maria Brown | P567890 | TK301 | Astana International Airport | Astana | Istanbul Airport
6 | Turkish Airlines | 22F, 22G | 150000.00
(6 rows)

airport=# SELECT * FROM Booking_receipt_1NF;
booking_id | passenger_full_name | passenger_passport_number | flight_number | departure_airport_name | departure_city | arrival_airport_name
-----
1 | Ali Friend | N123456 | KC101 | Almaty International Airport | Almaty | Astana International Airport
1 | Ali Friend | N123456 | KC101 | Almaty International Airport | Almaty | Astana International Airport
2 | Aruzhan Family | N234567 | KC102 | Astana International Airport | Astana | Almaty International Airport
3 | Dias Work | N345678 | FS201 | Almaty International Airport | Almaty | Frankfurt Airport
4 | John Smith | P456789 | LH500 | Frankfurt Airport | Frankfurt | Astana International Airport
5 | Maria Brown | P567890 | TK301 | Astana International Airport | Astana | Istanbul Airport
6 | Turkish Airlines | 22F, 22G | 150000.00
(6 rows)

airport=#

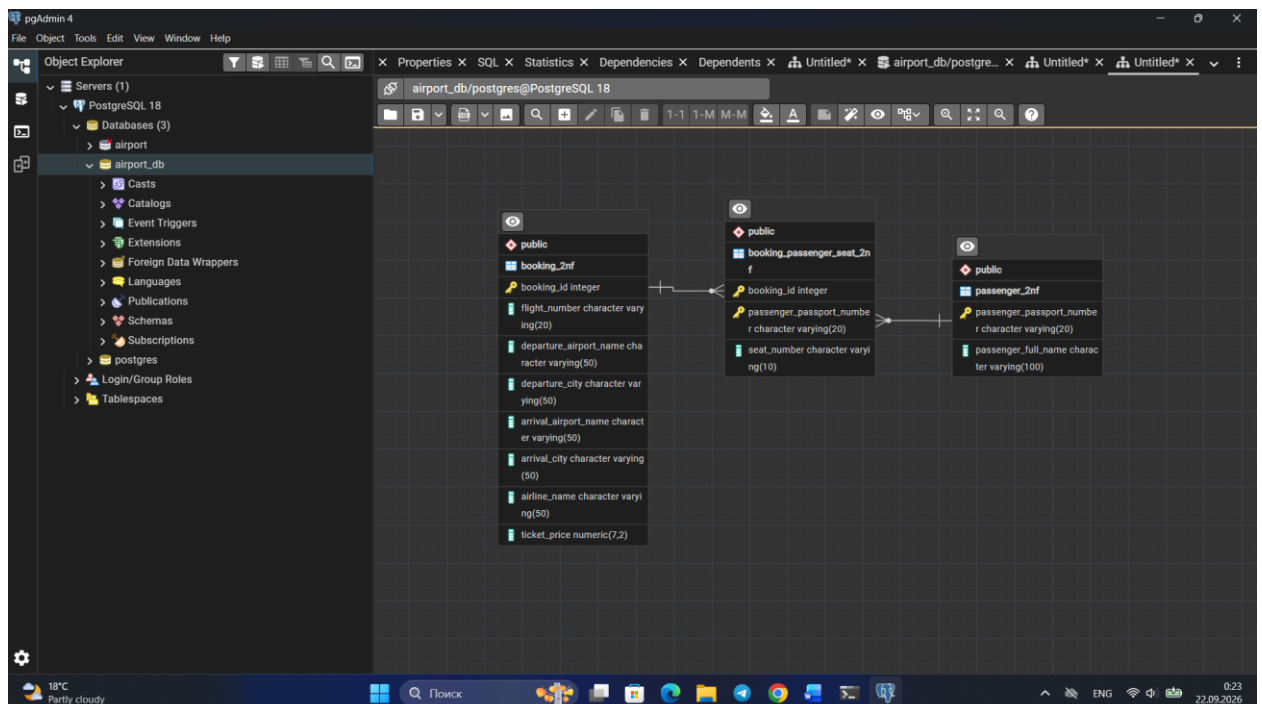
```



To bring the table to 1NF, I removed the list of seats (seat_numbers) from a single column. Now, each row contains only one atomic seat value (seat_number). Since one booking can have multiple passengers, the primary key is a composite key: (booking_id, passenger_passport_number).

- **Table:** Booking_receipt_1NF
- **Primary Key (PK):** (booking_id, passenger_passport_number)
- **Foreign Keys (FK):** None

2NF:



2NF: To reach 2NF, I removed partial functional dependencies. Attributes like passenger name depended only on passenger_passport_number, not on the whole primary key. I split the table into three new tables: Passenger_2NF, Booking_2NF, and a junction table Booking_Passenger_Seat_2NF.

- **Tables and Keys:**

1. **Passenger_2NF**

- **Primary Key (PK):** passenger_passport_number

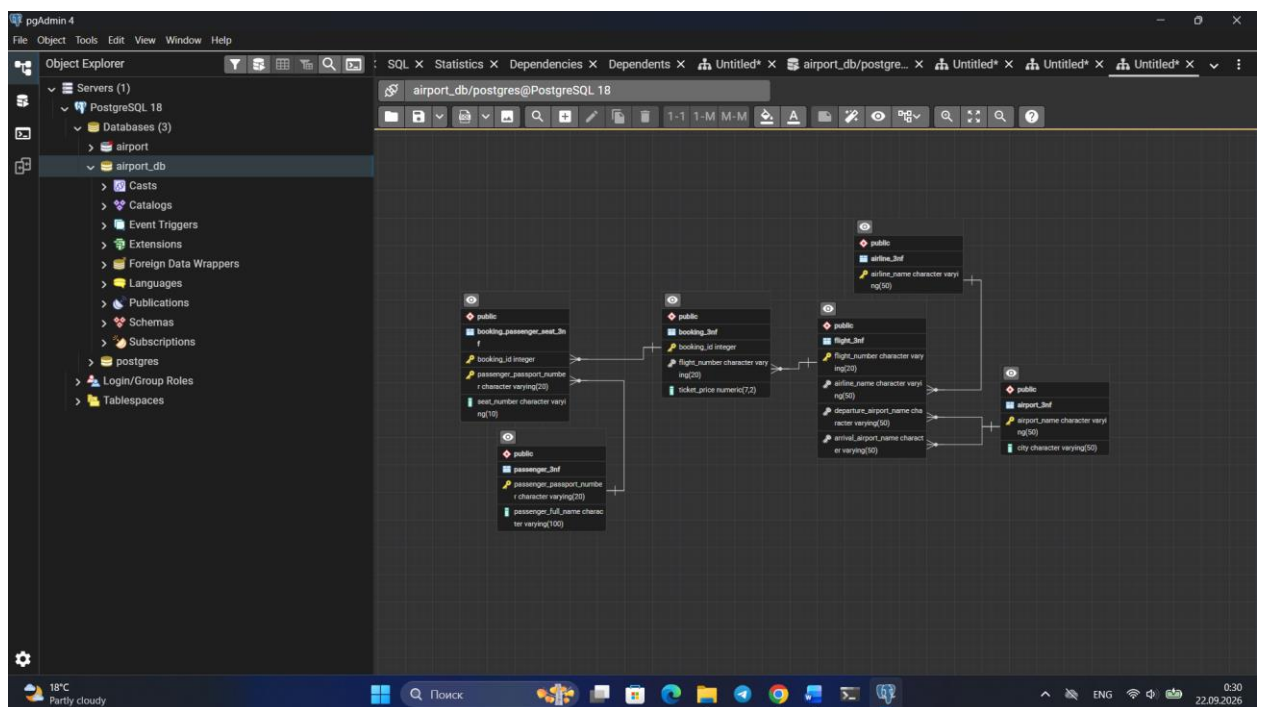
2. **Booking_2NF**

- **Primary Key (PK):** booking_id

3. **Booking_Passenger_Seat_2NF**

- **Primary Key (PK):** (booking_id, passenger_passport_number)
- **Foreign Keys (FK):**
 - booking_id → Booking_2NF(booking_id)
 - passenger_passport_number → Passenger_2NF(passenger_passport_number)

3NF:



3NF: To reach 3NF, I removed transitive functional dependencies where non-key attributes depended on other non-key attributes (for example, city depended on airport name, and

airline name depended on flight number). I created separate tables for Airport_3NF, Airline_3NF, and Flight_3NF.

- **Tables and Keys:**

1. **Airport_3NF**

- **Primary Key (PK):** airport_name

2. **Airline_3NF**

- **Primary Key (PK):** airline_name

3. **Flight_3NF**

- **Primary Key (PK):** flight_number
- **Foreign Keys (FK):** airline_name, departure_airport_name, arrival_airport_name

4. **Passenger_3NF**

- **Primary Key (PK):** passenger_passport_number

5. **Booking_3NF**

- **Primary Key (PK):** booking_id
- **Foreign Keys (FK):** flight_number

6. **Booking_Passenger_Seat_3NF**

- **Primary Key (PK):** (booking_id, passenger_passport_number)
- **Foreign Keys (FK):** booking_id, passenger_passport_number

TASK 5.

I created the 3NF database schema using CREATE TABLE statements with primary and foreign keys. Then, I populated the tables with 5 rows of sample data using INSERT queries.

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (3)
 - airport
 - airport_db
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

SQL

```
CREATE TABLE Booking_Passenger_Seat_3NF (  
    booking_id INT NOT NULL,  
    passenger_passport_number VARCHAR(20) NOT NULL,  
    seat_number VARCHAR(10) NOT NULL,  
    PRIMARY KEY (booking_id, passenger_passport_number),  
    FOREIGN KEY (booking_id) REFERENCES Booking_3NF(booking_id),  
    FOREIGN KEY (passenger_passport_number) REFERENCES Passenger_3NF(passenger_passpo  
);  
  
INSERT INTO Airport_3NF (airport_name, city) VALUES  
('Astana International', 'Astana'),  
('Almaty International', 'Almaty'),  
('Heathrow', 'London'),  
('JFK International', 'New York'),  
('Charles de Gaulle', 'Paris');  
  
INSERT INTO Airline_3NF (airline_name) VALUES  
('Air Astana'),  
('FlyArystan'),  
('British Airways'),  
('Delta Air Lines'),  
('Air France');
```

Data Output Messages Notifications

ERROR: повторяющееся значение ключа нарушает ограничение уникальности "airport_3nf_pkey"
Ключ "(airport_name)=(Astana International)" уже существует.

ОВИЕКА: повторяющееся значение ключа нарушает ограничение уникальности "airport_3nf_pkey"
SQL state: 23505
Detail: Ключ "(airport_name)=(Astana International)" уже существует.

Total rows: Query complete 00:00:00.031 CRLF Ln 35, Col 23

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (3)
 - airport
 - airport_db
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

SQL

```
SELECT * FROM Booking_3NF;
```

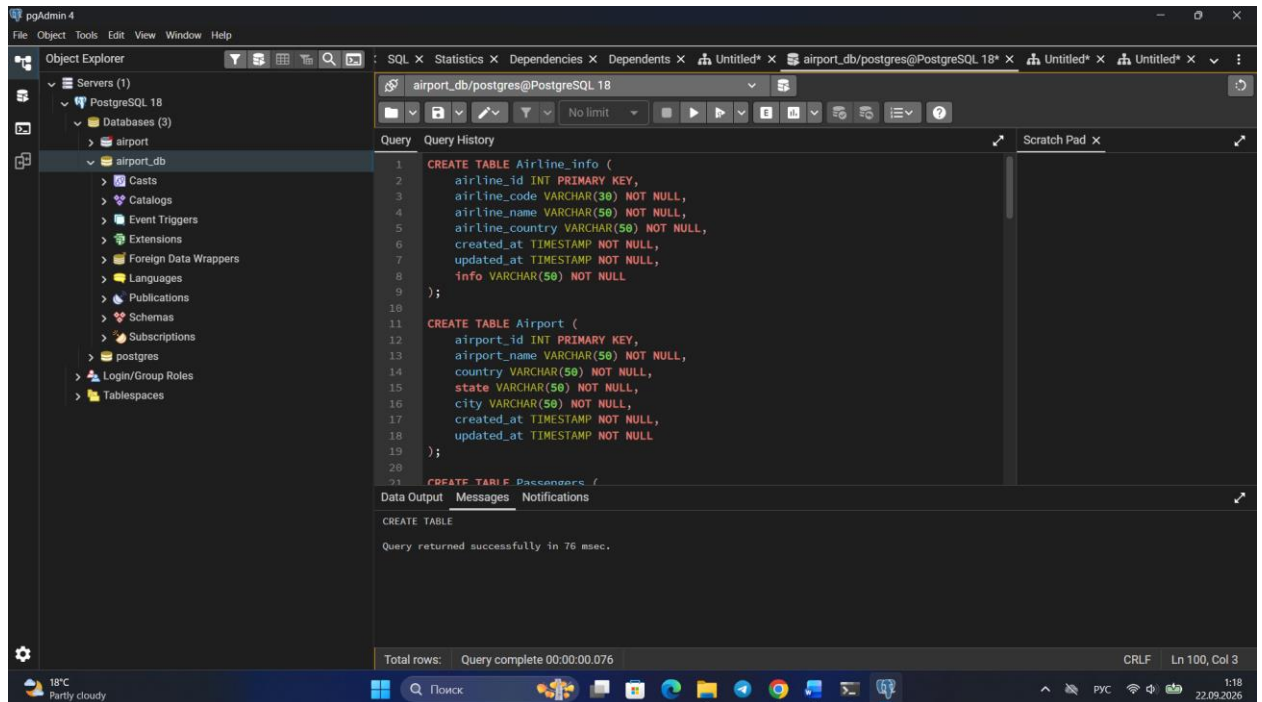
Data Output Messages Notifications

	booking_id [PK] integer	flight_number character varying (20)	ticket_price numeric (7,2)
1	101	KC-701	150.00
2	102	KC-802	450.00
3	103	FA-101	80.00
4	104	BA-202	600.00
5	105	AF-303	550.00

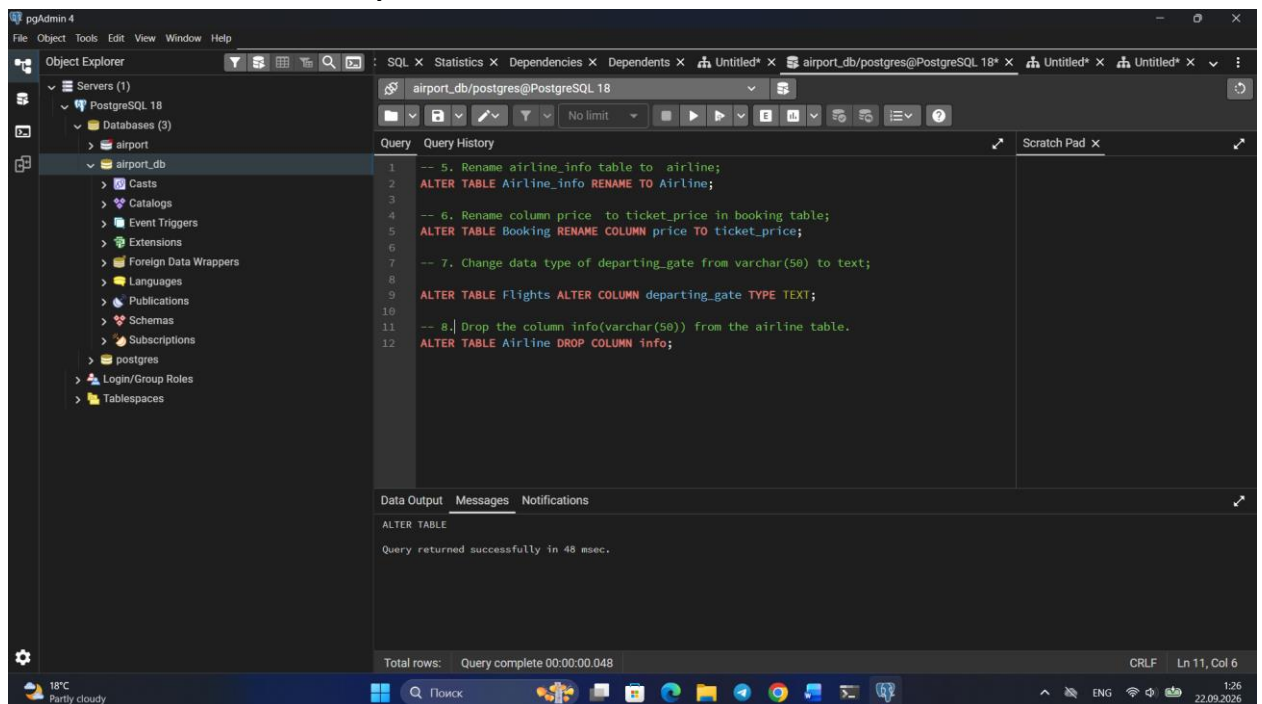
Total rows: 5 Query complete 00:00:00.268 CRLF Ln 1, Col 27

DDL

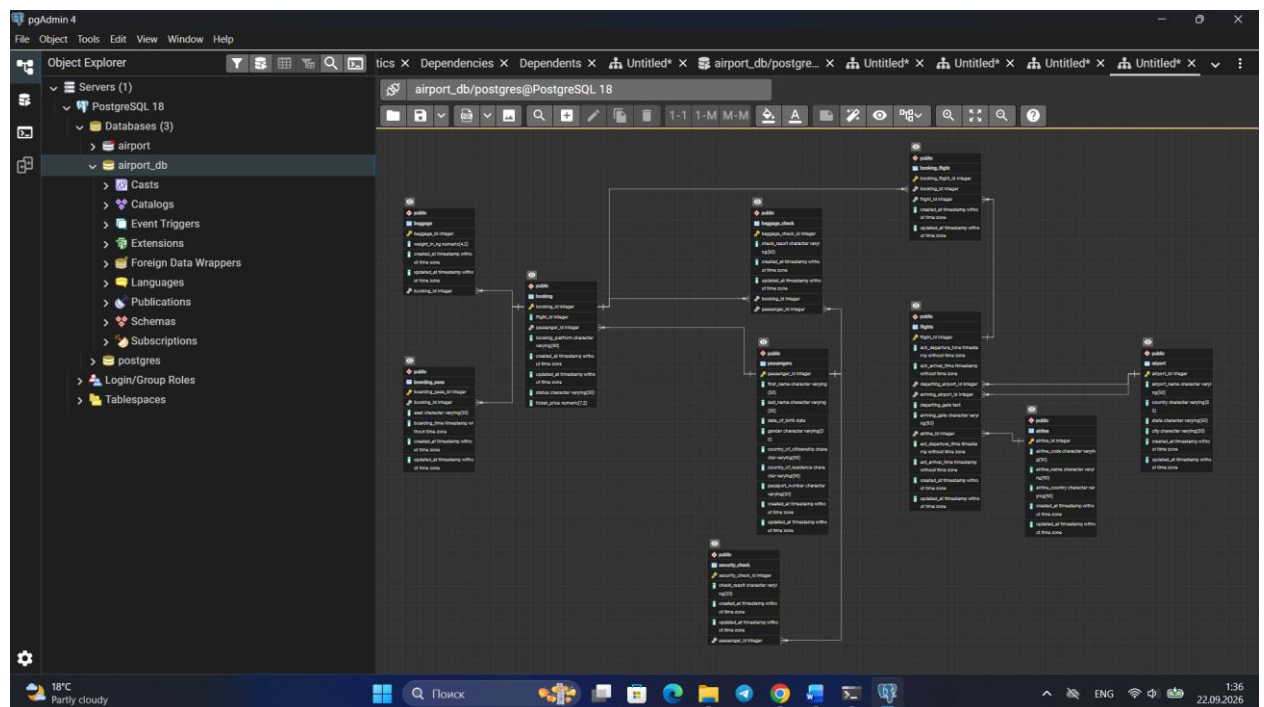
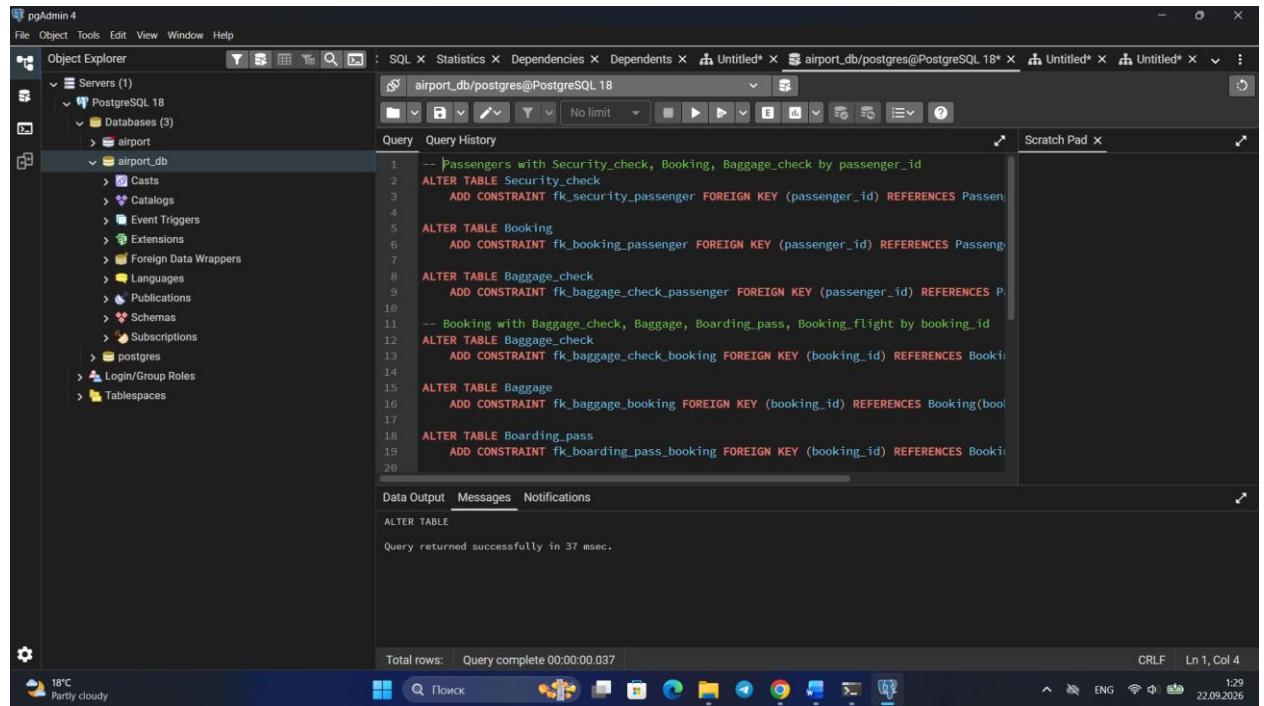
1. Creating all tables with PRIMARY KEY and NOT NULL constraints.



2. Modification of tables as per the task



3. Adding foreign keys (Relationships)



Task 4. 3NF Analysis for the Flights Table

Task Statement:

Check whether the Flights table from the DDL section is in 3NF. State the primary key, functional dependencies, and explain if any transitive dependency exists between non-key attributes.

Answer / Solution:

1. Current Primary Key (PK):

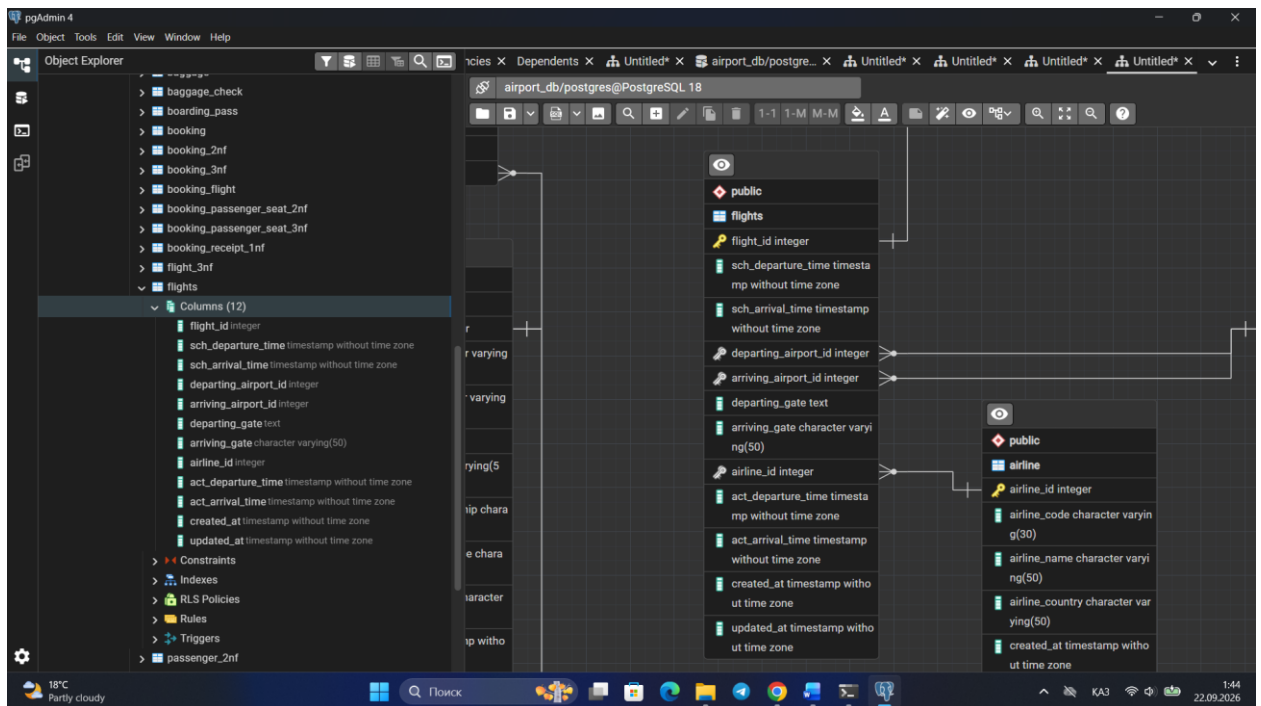
- **flight_id**

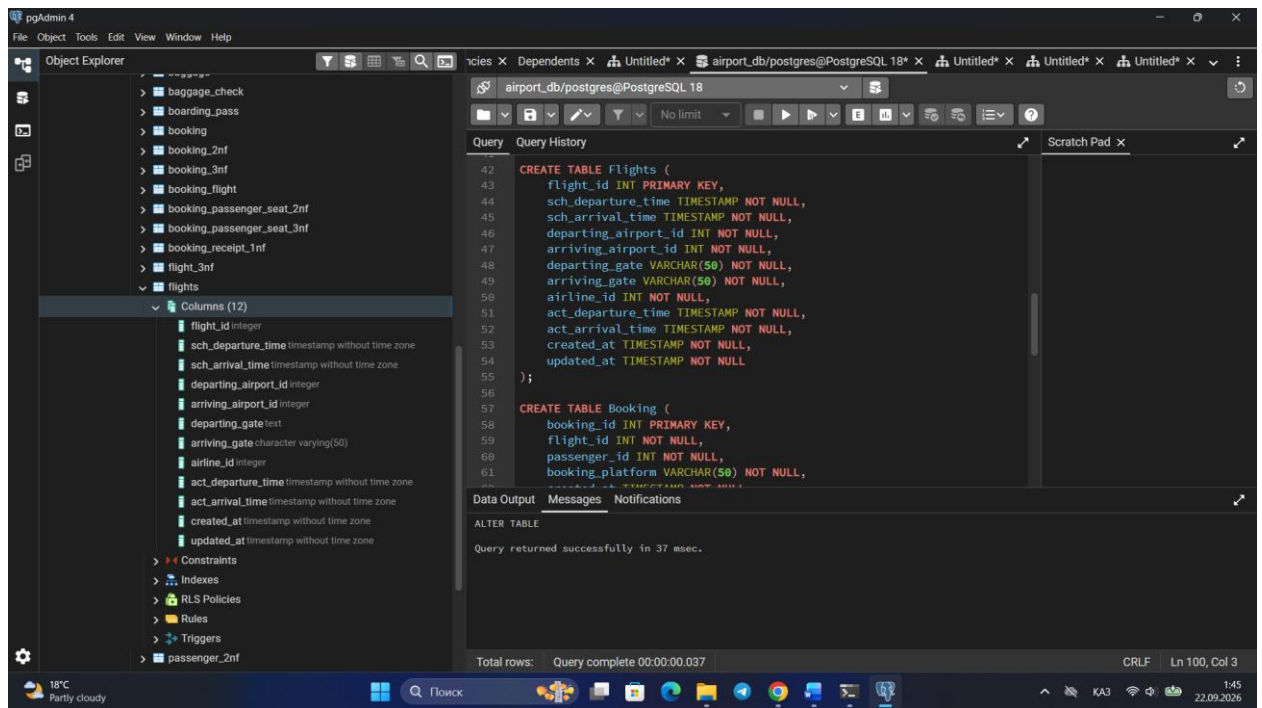
2. Functional Dependencies (FDs):

- **flight_id** → **sch_departure_time**, **sch_arrival_time**, **departing_airport_id**, **arriving_airport_id**, **departing_gate**, **arriving_gate**, **airline_id**, **act_departure_time**, **act_arrival_time**, **created_at**, **updated_at**

3. 3NF Analysis & Conclusion:

- Is the table in 3NF? Yes, the Flights table is in 3NF (Third Normal Form).
- Explanation:
 - All non-key attributes depend directly on the primary key (**flight_id**).
 - The attributes **departing_airport_id**, **arriving_airport_id**, and **airline_id** are Foreign Keys (FKs) that reference other tables (Airport and Airline).
 - There are no transitive dependencies between non-key attributes. Non-key attributes (like **departing_gate** or **sch_departure_time**) do not determine each other or depend on the foreign key IDs within this table.





Conclusion

In this laboratory work, I successfully normalized the Booking_receipt table step by step from 1NF to 3NF, removing non-atomic values, partial dependencies, and transitive dependencies. I analyzed the Flights table and confirmed it is in 3NF. Finally, I implemented the 3NF schema and the full airport database DDL in PostgreSQL with primary keys, foreign keys, and sample data.